국방 AI 경진대회
Military AI CONtest

MAICON 2026 MILITARY AI CONTEST

2026 국방 AI 경진대회 본선

2차 사전교육 자료대회

[개발운용]

주최 : 대한민국 국방부

주관 : 정보통신기획평가원 | 한국오픈소스협회

본선대회 협력업체 : 주식회사 리쿠스

※ 본 자료의 외부 공유 및 무단 복제, 배포를 금합니다.

개발환경

※ 본 자료의 외부 공유 및 무단 복제, 배포를 금합니다.

대회용 로봇(UGV) 소개

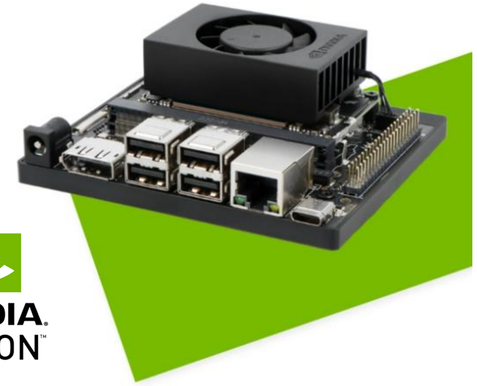


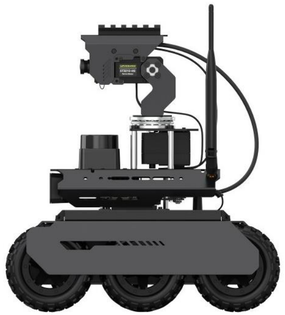
전술형 UGV(Unmanned Ground Vehicle) 로봇 플랫폼

- 모델명 : WINGBOT-UGV

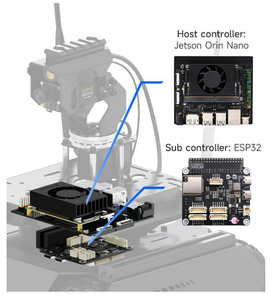
국방 무인전투체계를 모티브로 한 저중심(Low Profile)·모듈형(Modular) UGV 플랫폼으로
AI 기반 자율주행, 표적 탐지 및 레이저 조준 임무 수행에 최적화된 구조를 적용
주요장치 : Jetson Orin Nano 8G, LiDAR360도, Depth Camera, 6WD Wheel, ROS2, 레이저장치, 2축틸트

- **기술적 사양**
  - NVIDIA Jetson Orin Nano기반 전용 AI컴퓨팅 탑재
  - 2축 Pan-Tilt Auto Tracking 카메라
  - 레이저 발광/수광 시스템
  - LiDar기반 자율주행
  - RGB-D Depth Camera 적용
  - 6WD 고안정성 구동 플랫폼
  - 모듈형 센서 확장 구조
  - ROS2기반 AI 소프트웨어 플랫폼

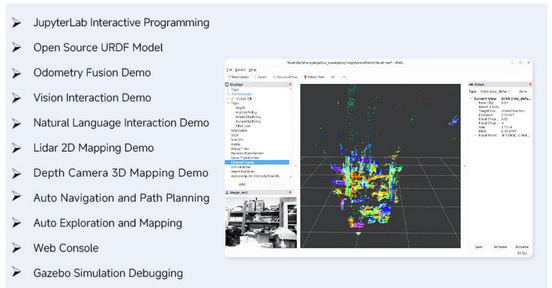














대회용 로봇(UGV) 소개



❖ 하드웨어 사양

항목	사양
메인 컴퓨팅	NVIDIA Jetson Orin Nano 8GB
저장공간	M.2 SSD 256GB
통신	Wi-Fi 5GHz, Wi-Fi 2.4GHz
카메라	160도, 5MP
전원장치	UPS 모듈
라이다센서	D500 Lidar 2D Mapping
Depth Camera	OAK Stereo Camera
배터리	12.6V, 3S2P 6셀구성, 18650(25R 30Q 35E)
팬틸트	팬틸트(ST3215 Servo), Horizontal 100도, Vertical 30도
모터제어기	ESP32제어기(Motor, Pan-tilt, LED, OLED, IMU, Battery)
모터	메탈기어모터
센서	IMU
출력장치	레이저, LAMP, LED(WS2812), OLED, 스피커
무게	약 2.9kg
제품크기	250(가로) x 230(세로) x 320(높이)mm

❖ 소프트웨어 사양

항목	사양
운영체제(OS)	Ubuntu 22.04
미들웨어(ROS)	ROS2-HUMBLE-LTS(Docker기반)
개발언어	Python, C++
ROS실행환경	ROS2 Docker
SLAM/Mapping	Gmapping 2D, Cartographer 2D, RTAB-Map 3D 지원
웹 기반 개발 도구	Jypyter Lab(http://<IP>:8888
API	상태값, 모터제어, IMU, OLED, 레이저, 라이다, 카메라 등

기본 구성



보관 케이스



임무용 UGV 로봇



배터리 충전기(12.6V 2A)



HDMI to DP 케이블



Jetson보드 어댑터

배터리 충전방법

STEP 1. 충전 준비

- 로봇의 모터 구동이 완전히 정지되었는지 확인합니다.
- 충전기 사양이 DC 12.6V / 2A인지 확인합니다.

STEP 2. 충전기 연결

- 제공된 충전기의 DC 커넥터를 충전 포트에 연결합니다.
- 충전기 표시등을 통해 충전 상태를 확인합니다.

STEP 3. 충전 진행

- 충전 중에도 UGV 로봇의 전원을 켜고 사용할 수 있습니다.
- 단, 충전 케이블이 연결된 상태에서는 모터를 구동하지 않습니다.
- Jetson Orin 및 Depth 카메라 등이 동작하면 전력 소모로 인해 충전 시간이 길어질 수 있습니다.
- 빠른 충전을 위해서는 가급적 로봇 전원을 끄거나 불필요한 장치의 사용을 중지하는 것을 권장합니다.

STEP 4. 충전 완료

- 충전기의 충전 완료 표시를 확인합니다. (OLED의 배터리 잔량 표시)

❖ UGV 로봇은 배터리 교체가 아닌 **충전 방식으로 운영**되며, 대회 준비기간 및 본선대회 중 배터리 부족에 따른 별도 교체는 불가합니다. 따라서 원활한 임무 수행을 위해 배터리 잔량을 수시로 확인하고, 불필요한 전력 소모를 최소화하는 등 효율적인 배터리 관리 및 전력 사용 최적화가 필요합니다. (불필요한 AI연산 최소화 및 Depth 카메라 사용 등)

❖ 배터리 사용시간 안내: 배터리 사용시간은 로봇의 운용 환경 및 장치별 사용량에 따라 달라지며, 특히 모터 구동부, Jetson Orin Nano(AI 연산), 팬틸트 서보모터, Depth 카메라 등의 동작 상태에 따라 전력 소모량이 크게 증가할 수 있습니다.

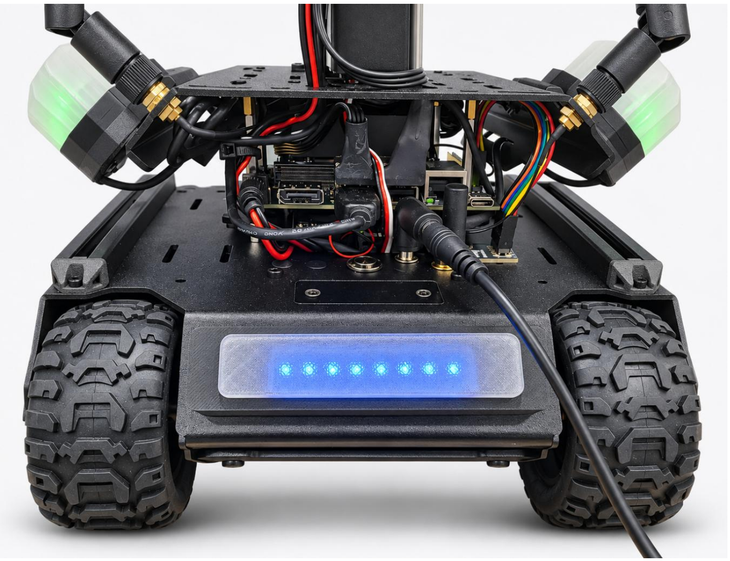
❖ 주의사항

- **배터리 충전에는 반드시 대회에서 제공한 전용 충전기(12.6V / 2A)만 사용합니다.**
- **Jetson Orin 전용 어댑터는 배터리 충전용이 아니므로 충전 포트에 연결하지 않습니다**
- **배터리는 충전방식으로 교체 불가(본선임무 수행을 위해 배터리 최적화 필요)**

❖ 배터리 기본 사양

항목	사양
배터리 종류	18650 리튬이온 (Li-ion)
배터리 구성	3S2P(3직렬 x 2병렬)
배터리 수량	총 6셀
전압	12.6V
전용 충전기	DC 12.6V 12A
배터리 사용시간	약40분(최대 소비전력)
충전 시간	100% 약 3시간 내외 (80%까지 약 1시간30분)

❖ UGV로봇 배터리 충전에는 지급된 전용 충전기를 사용합니다.

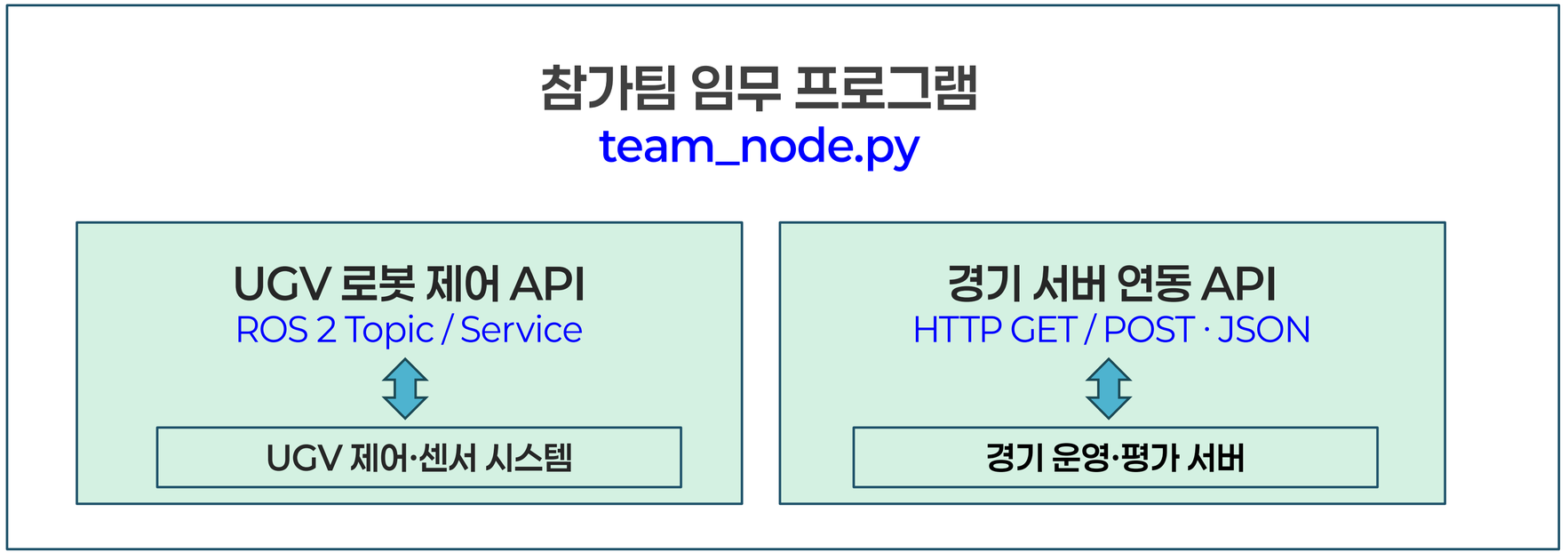


배터리 충전 방법

임무수행 API

UGV 임무 프로그램 및 API 통신 구조

참가가팀은 team_node.py를 기반으로 임무 수행 알고리즘을 개발하며, 로봇 제어와 경기 서버 통신을 위해 두 가지 API를 활용



- **UGV 로봇 제어 API**: ROS 2를 이용하여 로봇을 움직이고 센서 정보를 수집
- **경기 서버 연동 API**: HTTP 통신을 이용하여 경기 서버로부터 임무를 받고 수행 결과를 보고
- **참가팀 임무 프로그램**: 두 API를 연계하여 임무 판단, 자율주행, 표적 인식 및 결과 보고 등의 기능을 구현

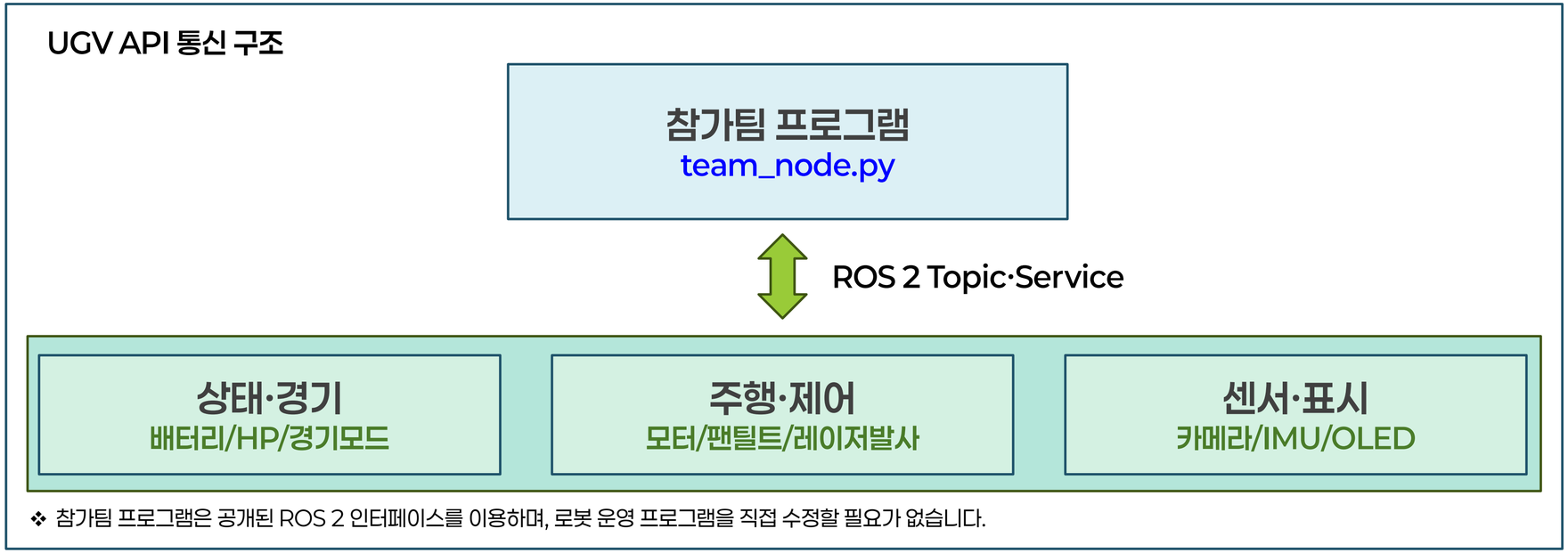
2026국방AI경진대회

UGV 로봇 제어 API (ROS 2)

※ 본 자료의 외부 공유 및 무단 복제, 배포를 금합니다.

UGV ROS2 API 구조

임무용 UGV로봇은 ROS 2 기반의 Topic 및 Service 인터페이스를 통해 로봇 상태 확인, 주행, 카메라 영상 수신, 팬틸트 제어, 레이저 발사 등의 기능을 제공.
참가자는 기본 제공 API를 활용하여 자율주행, 영상 인식, 표적 추적 등 임무 알고리즘을 개발할 수 있습니다.



❖ 참가팀 프로그램은 공개된 ROS 2 인터페이스를 이용하며, 로봇 운영 프로그램을 직접 수정할 필요가 없습니다.

ROS 2 API 통신 방식

구분	Topic (토픽)	Service (서비스)
통신 방식	데이터 발행·구독	요청·응답
주요 목적	센서 수신 및 주행 명령	조준 및 발사 명령
주요 API	/mil/state, /cmd_vel, /image_raw	/mil/gimbal, /mil/fire
특징	실시간 데이터 송수신	명령 처리 결과 확인

❖ 핵심 개념: Topic은 데이터를 지속적으로 송수신하고, Service는 특정 기능을 요청한 후 처리 결과를 확인하는 방식

UGV ROS2 API 전체

참가팀 코드 (rclpy)
~/team/team_node.py

Topic — Subscribe 수신 · Publish 송신
상태·경기·위치·카메라·IMU 수신 / 주행·OLED 명령

Service 요청·응답
팬틸트 조준 · 레이저 발사

UGV
운영 프로그램 · HW

기능	Topic / Service	방식	핵심 값
상태 · 경기	/mil/state · /mil/match	Subscribe	hp · hp_max · battery_pct (2Hz) · mode(practice/mission) · cmd(경기 단계)
OLED	/mil/oled	Publish	4줄(0~3) · 한 줄 21자 · 0.5초 이상 간격 · 10초 뒤 기본 화면
팬틸트	/mil/gimbal	Service	pan · tilt −100~100 (pan + 오른쪽) · speed 1~270 · acc 1~254 → ok / locked
주행 · 위치	/cmd_vel · /odom	Pub · Sub	linear.x ±0.6 m/s · angular.z ±3 rad/s (+ 좌회전) · 10Hz 반복 · 위치 20Hz
레이저	/mil/fire	Service	1회 5초 발광 · 발사 시작 10초 후 재발사 → fired / empty / locked / reloading
카메라	/image_raw	Subscribe	640×480 BGR · 최대 30FPS (10FPS 이하 권장) · CvBridge 변환
IMU	/imu/data	Subscribe	자세(쿼터니언 → roll·pitch·yaw) · 각속도 · 가속도 · 20Hz

- Subscribe : 데이터받기
- Publish : 데이터 보내기
- Pub·Sub : 보내기/받기
- Service : 요청, 응답

API - 상태확인

UGV 로봇의 체력(HP), 배터리 잔량, 현재 모드 및 경기 진행 상태를 확인하고, 경기 상태 변화를 감시

필드	설명
hp / hp_max	현재 · 최대 체력
battery_pct	배터리 잔량 (%)
mode	practice(연습) · mission(경기)
cmd	경기 진행 단계
control	제어 가능 여부 — 코드에서 계산

```
# 제어 가능 여부 판단
control = m.mode == 'practice' \
        or m.cmd in ('start', 'resume')

HP: 4/4 | Battery: 95%
Mode: mission | Match State: standby
Match Control: Locked
```

cmd	설명	경기 모드
idle	경기 없음	잠금
standby	경기 시작 대기	잠금
start	경기 진행	**허용**
pause	경기 일시정지	잠금
resume	경기 재개	**허용**
report	경기 종료 후 보고	잠금
stop	경기 종료	잠금

- QoS depth=1 · TRANSIENT_LOCAL — 마지막 상태를 바로 받음
- 연습 모드는 단계별 잠금 없음 · **피격 후 잠금·재발사 대기는 적용**

API - 경기모드

구분	연습 practice	경기 mission
설정	기본 모드	대회 운영 시스템이 전환
탄약 · HP	자동 회복	**회복 없음**
주행·조준·발사	반복 실습 가능	**start · resume**에서만
대기·정지·보고	잠금 미적용	주행 무시 · 조준·발사 locked
공통 적용	발사 간격 10초 · 피격 후 잠금 · 팬틸트 범위	

- 경기 모드는 **참가자가 직접 바꿀 수 없음** — `/mil/match`로 확인
- 일시정지 시 레이저 꺼짐 · 모터·팬틸트 정지
- 재개 시 로봇이 스스로 움직이지 않음 → **주행·팬틸트 명령 다시 전송**
- OLED 표시로 현재 단계 확인 — 연습은 `[P-MODE]`

경기 단계 흐름 · 경기 모드 기준 · 초록 = 제어 허용

idle	standby	start	pause	resume	report	stop
—	[M-STBY]	[M-RUN]	[M-PAUS]	[M-RUN]	[M-REPT]	[M-END]
잠금	잠금	허용	잠금	허용	잠금	잠금

API - OLED 제어

/mil/oled — OledLine(line, text)

- 4줄 출력 line=0~3 · 한 줄 **최대 21자** (영문·숫자·기호)
- 전송 간격 **0.5초 이상** · 마지막 출력 **10초 후** 기본 화면 복귀
- 팀 상태·탐색 단계,시간 표시 등 디버깅에 활용

```
TEAM ALPHA
HP 4/4
BAT 95%
15:30:25 SEARCH
```

```
pub = node.create_publisher(OledLine, '/mil/oled', 10)
pub.publish(OledLine(line=0, text='TEAM ALPHA'))
time.sleep(0.5)  # 간격 유지
```

API - 팬틸트 제어

/mil/gimbal — 팬틸트 위치 제어

필드	설명	범위
pan	좌우 · **+ 오른쪽** · 1단위 0.5°	−100 ~ 100
tilt	상하 · **+ 위쪽** · 1단위 0.15°	−100 ~ 100
speed	이동 속도 (°/s)	1 ~ 270
acc	가속도 (권장 20)	1 ~ 254

```
req = Gimbal.Request()
req.pan, req.tilt = 0.0, 0.0
req.speed, req.acc = 80.0, 20.0
future = client.call_async(req)
rclpy.spin_until_future_complete(node, future, timeout_sec=3.0)
# → Gimbal: ok
```

ok명령 접수 **locked**제어 잠금 **No response**응답 실패·시간 초과

pan·tilt는 각도가 아닌 **위치값** · ok는 도착이 아니라 **접수**

API - 주행

필드	설명	범위
`linear.x`	전진(+) · 후진(−)	−0.6 ~ 0.6 m/s
`angular.z`	**좌회전(+)** · 우회전(−)	−3 ~ 3 rad/s
`position.x/y`	출발점 기준 전후 · 좌우 위치	m
`orientation`	방향 쿼터니언 → yaw 변환	x, y, z, w

방향 기준 · 위에서 본 UGV




```
msg = Twist()
msg.linear.x = 0.15      # 전진 0.15 m/s
msg.angular.z = 0.0
for _ in range(10):      # 10Hz × 1초
    pub.publish(msg)
    time.sleep(0.1)
pub.publish(Twist())     # 정지

# Position: X=0.012m, Y=0.001m | Yaw=1.8deg
```

주행 안전

- 속도 명령은 약 **10Hz로 반복 전송 — 0.5초** 끊기면 자동 정지
- 여러 노드에서 동시에 `/cmd_vel` 전송 금지 (명령 충돌)
- 실제 주행 전 **충전 케이블 분리** · 안전한 장소에서 저속 시험
- `/odom` 20Hz · `qos_profile_sensor_data` 사용

API - 레이저발사

요청 파라미터 없이 `Fire.Request()` 전송 → **1회 5초 발광**, 발사 시작 **10초 후** 재발사 가능. 응답의 `left_ms`로 남은 대기 확인

```
future = client.call_async(Fire.Request())
rclpy.spin_until_future_complete(node, future, timeout_sec=3.0)
result = future.result()
print('Fire:', result.message)     # fired
print('Remaining:', result.left_ms) # 10000 ms
```

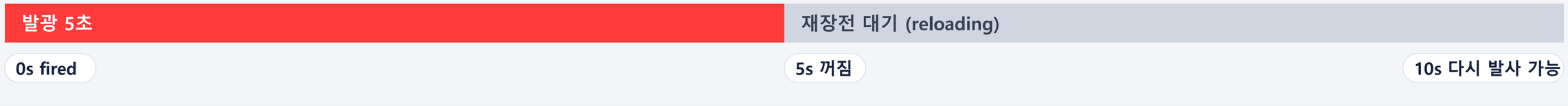
message	설명
fired	발사 성공
empty	탄약 없음
locked	발사 잠금 (대기·일시정지·보고·종료, 피격 후 10초)
reloading	재장전 대기 중

- 경기 모드는 탄약·HP **회복 없음** → 발사 타이밍 최적화가 핵심
- 피격 시 조준·발사 **10초 제한**
- HP 0이어도 탄약이 있고 다른 제한이 없으면 발사 가능
- 일시정지 시 레이저 꺼짐 · 재개 후 명령 **다시 전송**

⚠ 레이저 안전

사람의 눈에 직접 노출 시 **시력 손상** 위험 — 발사 전 주변 확인, 눈 방향 발사 금지

발사 주기 · 발사 시작부터 10초



API - 카메라·IMU

/image_raw — sensor_msgs/Image

- **640 × 480** · BGR(OpenCV) · 최대 30FPS — CPU 부담으로 **10FPS 이하 권장**
- `CvBridge().imgmsg_to_cv2(msg, 'bgr8')` → NumPy 배열
- 팬틸트 카메라 영상 — Depth 카메라는 이 토픽이 아님
- OpenCV 처리 · AI 객체 인식(M3)에 활용

```
sub = node.create_subscription(
    Image, '/image_raw', data.append,
    qos_profile_sensor_data)
frame = CvBridge().imgmsg_to_cv2(data[-1], 'bgr8')
print('Camera:', frame.shape)   # (480, 640, 3)
```

08 · /imu/data — sensor_msgs/

데이터	활용
roll / pitch (°)	로봇 기울기
yaw (°)	진행 방향
linear_acceleration	급격한 움직임 · 충격 감지
/odom position	이동 거리 · 현재 위치 추정

```
yaw = math.degrees(math.atan2(
    2*(q.w*q.z + q.x*q.y), 1 - 2*(q.y*q.y + q.z*q.z)))
g = math.sqrt(a.x**2 + a.y**2 + a.z**2) / 9.80665
# Roll: 0.2 | Pitch: -0.2 | Yaw: -34.5 | Acc: 1.01g
```

기울기·충격 감지 시 `/cmd_vel`에 `Twist()`(속도 0) 전송으로 **즉시 정지**

팀노드(team_node.py) 수행 기준

항목	내용
코드 경로	~/team/team_node.py
실행 시점	경기 시작 대기(standby) 때 운영 프로그램이 자동 실행
실행 형태	Python ROS 2 노드 · 하위 프로그램은 팀 노드가 실행·관리
경기 상태	/mil/match 구독
경기 종료	report 후 stop 수신 시 팀 노드 종료
오류 시	비정상 종료 시 경기당 최대 3회 자동 재실행
성능	step()은 0.1초 이내 · AI 추론은 별도 스레드

구현 체크리스트

1. `/mil/match` 구독 → mode · cmd로 제어 가능 여부 판단
2. `/mil/state`로 HP · 배터리 감시
3. 주행 명령 **10Hz 반복** · 종료·예외 시 `Twist()` 정지
4. 팬틸트 pan(+오른쪽) · 주행 angular.z(+왼쪽) **방향 확인**
5. 발사 응답 `fired · reloading · locked · empty` 처리
6. 카메라 10FPS 이하로 받아 **별도 스레드**에서 추론
7. resume 후 주행·팬틸트 명령 **다시 전송**
8. OLED로 팀 상태 표시해 현장 디버깅
9. 연습 모드에서 전 기능 시험 후 `~/team/`에 저장

경기 서버 연동 API (HTTP)

별첨) 경기서버연동API.html

※ 본 자료의 외부 공유 및 무단 복제, 배포를 금합니다.

국방 AI 경진대회
Military AI CONtest

MAICON 2026 MILITARY AI CONTEST

감사합니다.

※ 본 자료의 외부 공유 및 무단 복제, 배포를 금합니다.